

DOCUMENTACIÓN DE PYTEST

Autor: Andres David Pereira Puello

Rol: QA / TESTING / DS Specialist

Empresa: GBM

Fecha: 29/09/2025

1 Introducción

Pytest es un framework de testing en Python, enfocado en simplicidad y escalabilidad.

Permite escribir pruebas unitarias, funcionales e incluso de integración.

Ofrece un sistema de descubrimiento automático de pruebas, soporte para **fixtures**,

parametrización y **plugins** que facilitan la creación de un entorno de pruebas robusto.

2 Instalación

Asegúrate de tener Python y pip instalados.

En la terminal:

```
pip install pytest
```

Para confirmar que está instalado:

```
pytest --version
```

3 Estructura de Proyecto

```
project/  
├── app.py
```

```
|— utils.py
|— tests/
|   |— test_app.py
|   |— test_utils.py
|— requirements.txt
```

✓ Buenas prácticas:

- Coloca los tests en una carpeta `tests/`.
- Nombra los archivos con el prefijo `test_`.

4 Funciones a Probar

Ejemplo en `app.py`:

```
def suma(a, b):
    return a + b

def es_par(numero):
    return numero % 2 == 0
```

5 Creación de Tests

Ejemplo en `tests/test_app.py`:

```
from app import suma, es_par

def test_suma():
    assert suma(2, 3) == 5
    assert suma(-1, 1) == 0

def test_es_par():
    assert es_par(4) is True
    assert es_par(5) is False
```

6 Ejecución de Pruebas

En la terminal:

```
pytest -v
```

  significa "verbose", muestra más detalles de cada test.

Salida esperada:

```
tests/test_app.py::test_suma PASSED
tests/test_app.py::test_es_par PASSED
```

7 Uso de Fixtures

```
import pytest

@pytest.fixture
def numeros():
    return [1, 2, 3]

def test_lista_no_vacia(numeros):
    assert len(numeros) > 0
```

8 Parametrización

```
import pytest
from app import es_par

@pytest.mark.parametrize("numero, esperado", [(2, True), (3, False), (10, True)])
def test_es_par_parametrizado(numero, esperado):
    assert es_par(numero) == esperado
```

9 Mejores Prácticas

- Usa nombres descriptivos para los tests.

- Divide los tests en unitarios, funcionales e integración.
- Ejecuta los tests en cada commit o pipeline CI/CD.
- Usa `pytest-cov` para medir la cobertura: